

LAB 1 REPORT

Course: Database and Web Application Development
Lecturers: Nguyen Thanh Binh, Nguyen Phan Thao Nguyen

Student ID: 25560052 Student Name: Lê Minh Tân Class: CSBU109.R11.KHBC

Github: <https://github.com/onecot/csbul09-labs/tree/main/lab1>

Table of Contents

(Lab 1) Exercise 2: Design a Schema & Execute SQL DDL/DML on a MySQL Server	2
1. Extend the Database Structure (DDL).....	2
2. Data Manipulation & SQL Queries (DML & DQL)	3
• Q1 (Insert sample data):	3
• Q2 (Filter & search products):	3
• Q3 (Table joins - JOIN):	4
• Q4 (Revenue statistics - GROUP BY & AGGREGATE):	4
(Lab 1) Exercise 3: Working with MongoDB NoSQL Documents Using Mongo Compass / Shell	5
Q1 (Insert varied data)	5
Q2 (Conditional & nested-document queries).....	6
Q3 (Array update).....	7
Q4 (Aggregation Framework statistics).....	8
(Lab 1B) Exercise 2: Connecting to and Querying MySQL in Node.js (mysql2 Driver)	9
Question 1 (Filter & sort)	9
Question 2 (Wildcard / LIKE search)	10
Question 3 (Aggregate functions)	11
Question 4 (Category statistics - GROUP BY & HAVING).....	12
(Lab 1B) Exercise 3: NoSQL Data Modeling with Mongoose ODM in Node.js	13
Question 1 (Custom validation with Regex):.....	13
Question 2 (Virtual properties & formatting):	14
Question 3 (Static methods):	15
Question 4 (Instance methods / soft delete):	16
Question 5 (Middleware hooks - pre-save & query hook):	17

LAB 1: DEVELOPMENT TOOLS OVERVIEW, ENVIRONMENT SETUP & SQL DATABASE DESIGN

(Lab 1) Exercise 2: Design a Schema & Execute SQL DDL/DML on a MySQL Server

1. Extend the Database Structure (DDL)

Create **orders** table:

```
1      USE ecommerce_db;
2
3      -- E1. Extended the Database Structure (DDL)
4      -- E1.1. Create the orders table
5  ✓ CREATE TABLE IF NOT EXISTS orders (
6      id INT AUTO_INCREMENT PRIMARY KEY,
7      user_id INT NOT NULL,
8      total_amount DECIMAL(10, 2) NOT NULL,
9      status ENUM('pending', 'completed', 'cancelled') DEFAULT 'pending',
10     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
11     FOREIGN KEY (user_id) REFERENCES users(id)
12 );
```

Create **order_items** table:

```
1      •USE ecommerce_db;
2
3      -- E1. Extended the Database Structure (DDL)
4      -- E1.2 Create the order_items table
5  ✓ •CREATE TABLE IF NOT EXISTS order_items (
6      id INT AUTO_INCREMENT PRIMARY KEY,
7      order_id INT NOT NULL,
8      product_id INT NOT NULL,
9      quantity INT CHECK (quantity > 0),
10     price DECIMAL(10, 2) NOT NULL,
11     FOREIGN KEY (order_id) REFERENCES orders(id),
12     FOREIGN KEY (product_id) REFERENCES products(id)
13 );
```

2. Data Manipulation & SQL Queries (DML & DQL)

• Q1 (Insert sample data):

Insert at least 3 orders into the orders table for different users.

```
14      INSERT INTO orders(user_id, total_amount, status) VALUES
15      (1, 27400000.00, 'completed'),
16      (2, 1800000.00, 'pending'),
17      (1, 8500000.00, 'completed')
```

Output	Result #1		Result #2	
id	user_id	total_amount	status	created_at
1	1	27400000.00	completed	09/30/2026
2	2	1800000.00	pending	09/30/2026
3	1	8500000.00	completed	09/30/2026

Insert the corresponding product details into the order_items table.

```
15      INSERT INTO order_items(order_id, product_id, quantity, price) VALUES
16      (1, 1, 1, 25000000.00),
17      (1, 2, 1, 2400000.00),
18      (2, 3, 1, 1800000.00);
19      (3, 2, 1, 8500000.00);
```

Output	Result #1		Result #2	
id	order_id	product_id	quantity	price
1	1	1	1	25000000.00
2	1	2	1	2400000.00
3	2	3	1	1800000.00
4	3	2	1	8500000.00

• Q2 (Filter & search products):

Write a **SELECT** statement to retrieve the list of products priced between 100,000 VND and 1,000,000 VND, sorted by price from highest to lowest (**ORDER BY ... DESC**).

```
3      • SELECT * FROM products
4      WHERE price BETWEEN 100000 AND 1000000
5      ORDER BY price DESC;
```

Output	Result #1		Result #2		
id	title	price	stock	category	created_at

*No products with price ranged between 100,000 and 1,000,000 in **products** table.

• **Q3 (Table joins - JOIN):**

Write an **INNER JOIN** statement linking the **users**, **orders**, **order_items** and **products** tables to display a detailed order report with the columns: Order ID | Customer Name | Product Name | Quantity | Unit Price | Order Status

```

8  ✓ • SELECT
9      orders.id AS 'Order ID',
10     users.full_name AS 'Customer Name',
11     products.title AS 'Product Name',
12     order_items.quantity AS 'Quantity',
13     order_items.price AS 'Unit Price',
14     orders.status AS 'Order Status'
15 FROM orders
16 INNER JOIN users ON orders.user_id = users.id
17 INNER JOIN order_items ON orders.id = order_items.order_id
18 INNER JOIN products ON order_items.product_id = products.id;

```

Output	Result #2	Result #3	Result #4	Result #5	Result #6
Order ID	Customer Name	Product Name	Quantity	Unit Price	Order Status
1	Nguyen Van Admin	Laptop Dell XPS 15	1	25000000.00	completed
1	Nguyen Van Admin	Keychron K2	1	2400000.00	completed
2	Tran Thi User	Logitech MX Master 3S	1	1800000.00	pending
3	Nguyen Van Admin	Keychron K2	1	8500000.00	completed

• **Q4 (Revenue statistics - GROUP BY & AGGREGATE):**

Write a statement to calculate the total revenue from all completed orders (**status = 'completed'**).

```

5  • SELECT SUM(total_amount) AS total_revenue
6  FROM orders
7  WHERE status = 'completed';

```

Output	Result #2
total_revenue	
35900000.00	

Write a statement to count the number of orders placed by each user (**GROUP BY user_id**).

```

5  • SELECT user_id, COUNT(*) AS total_orders
6  FROM orders
7  GROUP BY user_id;

```

Output	Result #2	Result #3
user_id	total_orders	
1	2	
2	1	

(Lab 1) Exercise 3: Working with MongoDB NoSQL Documents Using Mongo Compass / Shell

Q1 (Insert varied data)

```
> db.orders.insertMany([
  {
    order_code: "ORD-2026-003",
    customer_name: "Le Minh C",
    customer_email: "leminhc@gmail.com",
    total_amount: 41200000,
    status: "completed",
    items: [
      { product_name: "Macbook Pro M4 14 inch", quantity: 1, price: 33000000 },
      { product_name: "Magic Mouse", quantity: 1, price: 3000000 },
      { product_name: "Magic Keyboard", quantity: 1, price: 5200000 }
    ],
    created_at: new Date()
  },
  {
    order_code: "ORD-2026-004",
    customer_name: "Ly Hoang D",
    customer_email: "lyhoangd@gmail.com",
    total_amount: 3500000,
    status: "cancelled",
    items: [
      { product_name: "Keychron K7 Pro", quantity: 1, price: 3500000 }
    ],
    created_at: new Date()
  }
]);
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abccffc42faf41187077285'),
    '1': ObjectId('6abccffc42faf41187077286')
  }
}
```

Q2 (Conditional & nested-document queries)

Query for orders with a total amount of 5,000,000 VND or more AND a status of 'completed'.

```
db.orders.find({
  total_amount: { $gte: 5000000 },
  status: "completed"
})
{
  _id: ObjectId('6abccd5842faf41187077283'),
  order_code: 'ORD-2026-001',
  customer_name: 'Nguyen Van A',
  customer_email: 'nguyenvana@gmail.com',
  total_amount: 37500000,
  status: 'completed',
  items: [
    {
      product_name: 'Laptop Dell XPS 15',
      quantity: 1,
      price: 35000000
    },
    {
      product_name: 'Logitech MX Master 3S Mouse',
      quantity: 1,
      price: 2500000
    }
  ],
  created_at: ISODate('2026-09-30T08:50:32.177Z')
}
{
  _id: ObjectId('6abccffc42faf41187077285'),
  order_code: 'ORD-2026-003',
  customer_name: 'Le Minh C',
  customer_email: 'leminhc@gmail.com',
  total_amount: 41200000,
  status: 'completed',
  items: [
    {
      product_name: 'Macbook Pro M4 14 inch',
      quantity: 1,
      price: 33000000
    },
    {
      product_name: 'Magic Mouse',
      quantity: 1,
      price: 3000000
    },
    {
      product_name: 'Magic Keyboard',
      quantity: 1,
      price: 5200000
    }
  ],
  created_at: ISODate('2026-09-30T09:01:48.814Z')
}
```

Query for all orders that include the product “Logitech MX Master 3S Mouse” in the items array.

```
db.orders.find({
  "items.product_name": "Logitech MX Master 3S Mouse"
}).pretty();
{
  _id: ObjectId('6abccd5842faf41187077283'),
  order_code: 'ORD-2026-001',
  customer_name: 'Nguyen Van A',
  customer_email: 'nguyenvana@gmail.com',
  total_amount: 37500000,
  status: 'completed',
  items: [
    {
      product_name: 'Laptop Dell XPS 15',
      quantity: 1,
      price: 35000000
    },
    {
      product_name: 'Logitech MX Master 3S Mouse',
      quantity: 1,
      price: 2500000
    }
  ],
  created_at: ISODate('2026-09-30T08:50:32.177Z')
}
```

Q3 (Array update)

```
db.orders.updateOne(
  { order_code: "ORD-2026-002" },
  {
    $push: {
      items: {
        product_name: "XL Gaming Mouse Pad",
        quantity: 1,
        price: 200000
      }
    },
    $inc: {
      total_amount: 200000
    }
  }
);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Q4 (Aggregation Framework statistics)

Write an aggregation command (\$match, \$group) to calculate the total revenue from all completed orders ('completed').

```
db.orders.aggregate([
  {
    $match: { status: "completed" }
  },
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: "$total_amount" }
    }
  }
]);
{
  _id: null,
  totalRevenue: 78700000
}
```

Write an aggregation command to count the number of orders grouped by status ('status').

```
db.orders.aggregate([
  {
    $group: {
      _id: "$status",
      count: { $sum: 1 }
    }
  }
]);
{
  _id: 'cancelled',
  count: 1
}
{
  _id: 'completed',
  count: 2
}
{
  _id: 'processing',
  count: 1
}
```

LAB 1B: DATA MODELING & DESIGN JSON/XML, MYSQL & MONGODB WITH MONGOOSE ODM

(Lab 1B) Exercise 2: Connecting to and Querying MySQL in Node.js (mysql2 Driver)

Build **items** table with a foreign key linking to **categories**.

```
56 // Build 'items' table in store_db
57 async function build_items_table() {
58   try {
59     console.log('Connecting to MySQL Database...');
60
61     // Create the 'items' tables if it doesn't exist
62     await pool.execute(`
63       CREATE TABLE IF NOT EXISTS items(
64         id INT AUTO_INCREMENT PRIMARY KEY,
65         category_id INT,
66         item_name VARCHAR(100) NOT NULL,
67         price DECIMAL(10, 2) NOT NULL,
68         quantity INT CHECK (quantity > 0),
69         FOREIGN KEY (category_id) REFERENCES categories(id)
70       )
71     `);
72     console.log("-> 'items' table is ready.");
73   } catch (error) {
74     console.error('MySQL error:', error.message);
75   } finally {
76     await pool.end(); // Close connection
77   }
78 }
```

Question 1 (Filter & sort)

Write a query that retrieves products priced $\geq 500,000$ VND AND with **quantity** > 0 , sorted by price in descending order (**ORDER BY price DESC**).

```
146 // Question 1:
147 async function question1() {
148   try {
149     console.log('Question 1...');
150
151     const [rows] = await pool.execute(`
152       SELECT * FROM items
153       WHERE price >= 500000 AND quantity > 0
154       ORDER BY price DESC
155     `);
156     console.log(rows);
157     console.log("-> Retrieve succesfully!");
158   } catch (error) {
159     console.error('MySQL error:', error.message);
160   } finally {
161     await pool.end(); // Close connection
162   }
163 }
164
165 question1();
```

```

● tan@onecot inclass_exercises2_lab1b % node mysql_demo.js
◇ injected env (5) from .env
Question 1...
[
  {
    id: 14,
    category_id: 3,
    item_name: 'Smartphone',
    price: '5500000.00',
    quantity: 15
  }
]
-> Retrieve succesfully!

```

Question 2 (Wildcard / LIKE search)

Write a function that searches for products by a given keyword (using a prepared statement with the **LIKE** '%keyword%' operator). Retrieve the products whose name contains “Gaming” or “Wireless”.

```

165 // Question 2:
166 async function searchByKeyword(keyword) {
167   try {
168     const [rows] = await pool.execute(`
169       SELECT * FROM items
170       WHERE item_name LIKE ?
171       `, [`${keyword}%`]);
172     console.log(rows);
173     console.log(`-> Search ${keyword} items successfully!\n`);
174   } catch (error) {
175     console.error('MySQL error:', error.message);
176   }
177 }
178
179 async function question2() {
180   console.log('Question 2...');
181   console.log(`-> Search for Gaming items...`);
182   await searchByKeyword("Gaming");
183   console.log(`-> Search for Wireless items...`);
184   await searchByKeyword("Wireless");
185   await pool.end(); // Close connection
186 }
187
188 question2()

```

```

• tan@onecot inclclass_exercises2_lab1b % node mysql_demo.js
  ◇ injected env (5) from .env
  Question 2...
  -> Search for Gaming items...
  []
  -> Search Gaming items successfully!

  -> Search for Wireless items...
  [
    {
      id: 15,
      category_id: 3,
      item_name: 'Wireless Mouse',
      price: '250000.00',
      quantity: 40
    }
  ]
  -> Search Wireless items successfully!

```

Question 3 (Aggregate functions)

Calculate the **total stock quantity** (SUM(quantity)), **average price** (AVG(price)), and **total number of items** (COUNT(*)) across the entire **items** table.

```

188 // Question 3:
189 async function question3() {
190   try {
191     console.log('Question 3...');
192
193     const [rows] = await pool.execute(`
194       SELECT
195         SUM(quantity) AS total_stock_quantity,
196         AVG(price) AS avg_price,
197         COUNT(*) AS total_num_of_items
198       FROM items
199     `);
200     console.log(rows);
201     console.log("-> Calculated successfully!");
202   } catch (error) {
203     console.error('MySQL error:', error.message);
204   } finally {
205     await pool.end(); // Close connection
206   }
207 }
208
209 question3();

```

```

tan@onecot inclass_exercises2_lab1b % node mysql_demo.js
◊ injected env (5) from .env
Question 3...
[
  {
    total_stock_quantity: '455',
    avg_price: '666700.000000',
    total_num_of_items: 10
  }
]
-> Calculate successfully!

```

Question 4 (Category statistics - GROUP BY & HAVING)

For each category, calculate the total number of items and the total inventory value (**SUM(price * quantity)**). Only show categories whose total inventory value exceeds **10,000,000 VND** (using **HAVING**).

```

209 // Question 4:
210 async function question4() {
211   try {
212     console.log('Question 4...');
213
214     const [rows] = await pool.execute(`
215       SELECT
216         categories.id AS category_id,
217         categories.name AS category_name,
218         COUNT(*) AS number_of_items,
219         SUM(items.price * items.quantity) AS total_value
220       FROM items
221       INNER JOIN categories ON items.category_id = categories.id
222       GROUP BY categories.id, categories.name
223       HAVING SUM(items.price * items.quantity) > 10000000
224     `);
225     console.log(rows);
226     console.log("-> Calculated successfully!");
227   } catch (error) {
228     console.error('MySQL error:', error.message);
229   } finally {
230     await pool.end(); // Close connection
231   }
232 }
233
234 question4();

```

```

tan@onecot inclass_exercises2_lab1b % node mysql_demo.js
◊ injected env (5) from .env
Question 4...
[
  {
    category_id: 3,
    category_name: 'Electronics',
    number_of_items: 2,
    total_value: '92500000.00'
  },
  {
    category_id: 4,
    category_name: 'Clothing',
    number_of_items: 2,
    total_value: '15950000.00'
  }
]
-> Calculated successfully!

```

(Lab 1B) Exercise 3: NoSQL Data Modeling with Mongoose ODM in Node.js

Question 1 (Custom validation with Regex):

Add **phone** field and custom validator, using regular expression, to **userSchema**.

```
28   phone: {
29     type: String,
30     required: [true, 'Phone number is required'],
31     validate: {
32       validator: function (value) {
33         return /^(03|05|07|08|09)\d{8}$/.test(value);
34       },
35       message: 'Invalid Vietnamese phone number'
36     }
37   },
```

Try to insert a user with a badly formatted phone number to trigger the validation error.

```
103 // Question 1
104 async function insertBadlyPhone() {
105   try {
106     console.log("Question 1, try to insert badly formatted phone number...");
107     // CREATE a user
108     const newUser = await User.create({
109       fullName: "Nguyen Van A",
110       email: "anguyen@example.com",
111       phone: "011234567",
112       age: 22
113     });
114   } catch (error) {
115     console.error("Expected validation error:", error);
116   } finally {
117     await mongoose.connection.close();
118     console.log("-> Mongoose connection closed.");
119   }
120 }
121
122 insertBadlyPhone();
```

```
● tan@onecot inclass_exercises3_lab1b % node mongoose_demo.js
  ◊ injected env (1) from .env
Question 1, try to insert badly formatted phone number...
Expected validation error: ValidationError: User validation failed: phone: Invalid Vietnamese phone number
    at model.validate (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/document.js:2945:36)
    at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
    at async model.$_save (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/model.js:399:7)
    at async maybeTracedSave (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/model.js:676:7)
    at async model.create (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/model.js:2807:5)
    at async insertBadlyPhone (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/mongoose_demo.js:108:25) {
  errors: {
    phone: ValidatorError: Invalid Vietnamese phone number
        at SchemaString.doValidate (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/schemaType.js:1547:13)
        at model.validate (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/document.js:2923:20)
        at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
        at async model.$_save (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/model.js:399:7)
        at async maybeTracedSave (/Users/tan/Documents/STUDY/KHBC2025/3RD_TERM/CSBU109/labs/lab1/inclass_exercises3_lab1b/node_modules/mongoose/lib/model.js:676:7) {
      properties: [Object],
      kind: 'user defined',
      path: 'phone',
      value: '011234567',
      reason: undefined,
      Symbol(mongoose#validatorError): true
    }
  },
  _message: 'User validation failed'
}
-> Connected to MongoDB successfully via Mongoose ODM!
-> Mongoose connection closed.
```

Question 2 (Virtual properties & formatting):

Create virtual field **displayInfo**.

```
53   timestamps: true, // Automatically add createdAt and updatedAt fields
54   toJSON: { virtuals: true },
55   toObject: { virtuals: true }
56 }
57
58 // Question 2: Create a virtual field displayInfo
59 userSchema.virtual('displayInfo')
60   .get(function() {
61     return `${this.fullName} <${this.email}> [${this.role.toUpperCase()}]`;
62   })
```

Test virtual field **displayInfo**.

```
130 // Question 2
131 async function testDisplayInfo() {
132   try {
133     console.log("Question 2, test virtual field displayInfo...");
134     // CREATE a user
135     const newUser = await User.create({
136       fullName: "Nguyen Van A",
137       email: "anguyen@example.com",
138       phone: "0312345678",
139       age: 22
140     });
141
142     console.log('Display info:', newUser.displayInfo);
143     console.log('JSON user:', newUser.toJSON());
144   } catch (error) {
145     console.error("Mongoose validation/operation error:", error);
146   } finally {
147     await mongoose.connection.close();
148     console.log("-> Mongoose connection closed.");
149   }
150 }
151
152 testDisplayInfo();
```

```
tan@onecot inclass_exercises3_lab1b % node mongoose_demo.js
◊ injected env (1) from .env
Question 2, test virtual field displayInfo...
[Middleware Pre-save] Preparing to save user: Nguyen Van A
-> Connected to MongoDB successfully via Mongoose ODM!
Display info: Nguyen Van A <anguyen@example.com> [USER]
JSON user: {
  fullName: 'Nguyen Van A',
  email: 'anguyen@example.com',
  phone: '0312345678',
  age: 22,
  role: 'user',
  isActive: true,
  _id: new ObjectId('6abd01fc81c91f3333c592ca'),
  createdAt: 2026-09-30T12:35:08.832Z,
  updatedAt: 2026-09-30T12:35:08.832Z,
  __v: 0,
  displayInfo: 'Nguyen Van A <anguyen@example.com> [USER]',
  id: '6abd01fc81c91f3333c592ca'
}
-> Mongoose connection closed.
```

Question 3 (Static methods):

Add static method `findActiveByRole(roleName)`.

```
64 // Question 3: Add static method findActiveByRole(roleName)
65 userSchema.statics.findActiveByRole = function(roleName) {
66     return this.find({
67         role: roleName,
68         isActive: true
69     }).sort({ fullName: 1 });
70 };
```

Test the new static method.

```
160 // Question 3
161 async function testFindActiveByRole() {
162     try {
163         await User.deleteMany({});
164
165         await User.create([
166             {
167                 fullName: "Tran Thi Mai",
168                 email: "mai@example.com",
169                 phone: "0912345678",
170                 age: 24,
171                 role: "user",
172                 isActive: true
173             },
174             {
175                 fullName: "Le Minh Tan",
176                 email: "tan@example.com",
177                 phone: "0912345679",
178                 age: 22,
179                 role: "admin",
180                 isActive: true
181             },
182             {
183                 fullName: "Nguyen Van An",
184                 email: "an@example.com",
185                 phone: "0987654321",
186                 age: 25,
187                 role: "manager",
188                 isActive: false
189             }
190         ]);
191
192         const activeAdmins = await User.findActiveByRole("admin");
193         console.log("Active admins:", activeAdmins);
194     } catch (error) {
195         console.error("Question 3 error:", error.message);
196     } finally {
197         await mongoose.connection.close();
198     }
199 }
200
201 testFindActiveByRole();
```

```

• tan@onecot inclass_exercises3_lab1b % node mongoose_demo.js
◇ injected env (1) from .env
-> Connected to MongoDB successfully via Mongoose ODM!
[Middleware Pre-save] Preparing to save user: Tran Thi Mai
[Middleware Pre-save] Preparing to save user: Le Minh Tan
[Middleware Pre-save] Preparing to save user: Nguyen Van An
Active admins: [
  {
    _id: new ObjectId('6abd03b685527721ec6da5f5'),
    fullName: 'Le Minh Tan',
    email: 'tan@example.com',
    phone: '0912345679',
    age: 22,
    role: 'admin',
    isActive: true,
    createdAt: 2026-09-30T12:42:30.803Z,
    updatedAt: 2026-09-30T12:42:30.803Z,
    __v: 0,
    displayInfo: 'Le Minh Tan <tan@example.com> [ADMIN]',
    id: '6abd03b685527721ec6da5f5'
  }
]

```

Question 4 (Instance methods / soft delete):

Add `isDeleted` field.

```

52     isDeleted: {
53       type: Boolean,
54       default: false
55     }

```

Add instance method `softDelete()`.

```

76 // Question 4: Add instance method isDeleted
77 userSchema.methods.softDelete = async function() {
78   this.isDeleted = true;
79   this.isActive = false;
80   await this.save();
81   return this;
82 }

```

Test `softDelete()`.

```

213 // Question 4
214 async function testSoftDelete() {
215     try {
216         const user = await User.findOne({ email: "tan@example.com" });
217
218         if (!user) {
219             throw new Error("User not found");
220         }
221
222         const deletedUser = await user.softDelete();
223
224         console.log("Soft-deleted user:", {
225             fullName: deletedUser.fullName,
226             isDeleted: deletedUser.isDeleted,
227             isActive: deletedUser.isActive
228         });
229     } catch (error) {
230         console.error("Question 4 error:", error.message);
231     } finally {
232         await mongoose.connection.close();
233     }
234 }
235
236 testSoftDelete()

```

```

tan@onecot inclass_exercises3_lab1b % node mongoose_demo.js
◇ injected env (1) from .env
-> Connected to MongoDB successfully via Mongoose ODM!
[Middleware Pre-save] Preparing to save user: Le Minh Tan
Soft-deleted user: { fullName: 'Le Minh Tan', isDeleted: true, isActive: false }

```

Question 5 (Middleware hooks - pre-save & query hook):

Add password field.

```

38     password: {
39         type: String,
40         required: [true, 'Password is required']
41     },

```

Add password hashing hook & pre-find hook.

```

88 // Add a pre-save middleware hook
89 userSchema.pre('save', function() {
90     // Question 5: Hashing the password before saving
91     if (this.isModified('password')) {
92         this.password = `hashed_${this.password}`;
93     }
94
95     console.log(`[Middleware Pre-save] Preparing to save user: ${this.fullName}`);
96 });
97
98 // Question 5: Auto apply isDeleted: true filter when finding
99 userSchema.pre(/^find/, function() {
100     this.where({ isDeleted: false });
101 });

```

Test hooks.

```
251 // Question 5
252 async function testMiddlewareHooks() {
253   try {
254     console.log("Question 5, testing middleware hooks...");
255
256     await User.deleteMany({});
257
258     const activeUser = await User.create({
259       fullName: "Active User",
260       email: "active@example.com",
261       phone: "0912345678",
262       password: "active123",
263       age: 22,
264       role: "user"
265     });
266
267     const userToDelete = await User.create({
268       fullName: "Deleted User",
269       email: "deleted@example.com",
270       phone: "0987654321",
271       password: "delete123",
272       age: 25,
273       role: "user"
274     });
275
276     console.log("Hashed password:", activeUser.password);
277
278     await userToDelete.softDelete();
279
280     console.log("Soft-deleted flags:", {
281       isDeleted: userToDelete.isDeleted,
282       isActive: userToDelete.isActive
283     });
284
285     const visibleUsers = await User.find({});
286     const hiddenUser = await User.findOne({
287       email: "deleted@example.com"
288     });
289
290     console.log("Users returned by find:", visibleUsers.map(user => user.email));
291     console.log("Deleted user returned by findOne:", hiddenUser);
292   } catch (error) {
293     console.error("Question 5 error:", error.message);
294   } finally {
295     await mongoose.connection.close();
296     console.log("-> Mongoose connection closed.");
297   }
298 }
299
300 testMiddlewareHooks();
```

```
tan@onecot inclass_exercises3_lab1b % node mongoose_demo.js
◇ injected env (1) from .env
Question 5, testing middleware hooks...
-> Connected to MongoDB successfully via Mongoose ODM!
[Middleware Pre-save] Preparing to save user: Active User
[Middleware Pre-save] Preparing to save user: Deleted User
Hashed password: hashed_active123
[Middleware Pre-save] Preparing to save user: Deleted User
Soft-deleted flags: { isDeleted: true, isActive: false }
Users returned by find: [ 'active@example.com' ]
Deleted user returned by findOne: null
-> Mongoose connection closed.
```